

1. Kesimpulan Materi Abstraction

Abstraction adalah proses menyembunyikan detail implementasi dari pengguna dan hanya menampilkan fungsionalitas yang dibutuhkan.

Pengguna hanya tahu apa yang dilakukan objek, bukan bagaimana cara kerjanya.

Cara mencapai abstraction adalah dengan Abstract Class dan Interface

a. Abstract

adalah sebuah class yang dideklarasikan menggunakan keyword `abstract`. Abstract class memiliki sifat serupa dengan class biasa, namun dapat mengandung abstract method (method tanpa implementasi) dan tidak dapat diinstansiasi secara langsung. Untuk menggunakan abstract class, harus dibuat subclass (class turunan) yang mewarisi abstract class tersebut. Jika sebuah class biasa mewarisi abstract class, maka class tersebut wajib mengimplementasikan seluruh abstract method yang ada di dalam abstract class tersebut.

b. Interface

Interface adalah struktur yang mendukung konsep multiple inheritance, artinya sebuah class dapat mengimplementasikan lebih dari satu interface. Interface digunakan untuk mencapai loose coupling, yaitu keterkaitan antar class yang lebih longgar, sehingga meningkatkan fleksibilitas program. Interface dideklarasikan menggunakan keyword `interface`, dan biasanya berisi konstanta (`public static final` variable) serta method tanpa implementasi (abstract method). Sejak Java 8, interface juga dapat memiliki default method. Hubungan antara class dan interface adalah bahwa class harus mengimplementasikan method-method yang didefinisikan dalam interface, memungkinkan adanya pewarisan berganda di Java.\

2. Code Implementasi

a. Abstract Class Kendaraan

```
1 package abstraction;
2
3 abstract class Kendaraan {
4     String nama;
5
6     Kendaraan(String nama) {
7         this.nama = nama;
8     }
9
10    abstract void berjalan();
11
12    void info() {
13        System.out.println("Nama Kendaraan: " + nama);
14    }
15 }
```

b. Interface Mesin

```
1 package abstraction;
2 interface Mesin {
3     void hidupkanMesin();
4     void matikanMesin();
5 }
```

c. Class Mobil

```

1 package abstraction;
2 class Mobil extends Kendaraan implements Mesin {
3
4     Mobil(String nama) {
5         super(nama);
6     }
7
8     @Override
9     void berjalan() {
10         System.out.println(nama + " sedang berjalan di jalan raya.");
11     }
12
13     @Override
14     public void hidupkanMesin() {
15         System.out.println("Mesin " + nama + " dihidupkan.");
16     }
17
18     @Override
19     public void matikanMesin() {
20         System.out.println("Mesin " + nama + " dimatikan.");
21     }
22 }

```

d. Main Class

```

1 package abstraction;
2 public class Main {
3     public static void main(String[] args) {
4         Mobil mobilku = new Mobil("Toyota Avanza");
5
6         mobilku.info();
7         mobilku.hidupkanMesin();
8         mobilku.berjalan();
9         mobilku.matikanMesin();
10    }
11 }

```

e. Output

```

run:
Nama Kendaraan: Toyota Avanza
Mesin Toyota Avanza dihidupkan.
Toyota Avanza sedang berjalan di jalan raya.
Mesin Toyota Avanza dimatikan.
BUILD SUCCESSFUL (total time: 0 seconds)

```